

FC7xxx I2C Integration Manual

Rev.0.3

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0
0.3	2023/10/20	Updated for MCAL V0.3.0

Table of Contents

Revision History	2
Table of Contents	3
Chapter 1 Introduction	4
1.1 Introduction	4
Chapter 2 Building	5
2.1 Dependencies on Other Modules	5
2.2 Files Required for Compile	5
2.3 Add Plug-ins	6
Chapter 3 Memory	7
3.1 Sections in Memory Map	7
Chapter 4 Exclusive Area	9
Chapter 5 Interrupt Service Routine (ISR)	11
Chapter 6 Error Report	12
6.1 Det	12
6.2 Dem	12
Chapter 7 Function Calls	13
7.1 Function Calls during Startup	13
7.2 Function Calls during Shutdown	13
7.3 Function Calls during Wake-up	13
7.4 Function Calls during Runtime	13
Chapter 8 Other Requirements	14
8.1 Notification, Callback, Callout	14
8.2 Macros	14
Chapter 9 Integration Steps	15

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the I2C module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides the clock reference point for I2C module.
- Port: This module provides pin port configuration relevant to I2C buses.
- Common: This module is the basic module which used to choose the chip variant.
- DMA: This module provides the DMA channel reference point for I2C module.
- Dem: This module is used for reporting production relevant error status.

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for enabling Development error detection.

Module initialization dependency

- Mcu: To use the I2C module, the user needs to initialize MCU module first.
- Port: To use I2C module, SCL、SDA pins for each I2C should be configured.
- DMA: To use I2C module DMA function, the user needs to initialize DMA module first.

2.2 Files Required for Compile

I2C module files:

- _MCAL/Src/I2c/Src/CDD_I2c.c
- _MCAL/Src/I2c/Src/CDD_I2c_Hw.c
- _MCAL/Src/I2c/Src/CDD_I2c_Irq.c
- _MCAL/Src/I2c/include/CDD_I2c.h
- _MCAL/Src/I2c/include/CDD_I2c_Hw.h
- _MCAL/Src/I2c/include/CDD_I2c_HwType.h
- _MCAL/Src/I2c/include/CDD_I2c_RegOps.h
- _MCAL/Src/I2c/include/CDD_I2c_Reg.h
- _MCAL/Src/I2c/include/CDD_I2c_Types.h
- _MCAL/Src/I2c/include/I2c_MemMap.h
- _MCAL/Src/I2c/include/I2c_version.h
- _MCAL_generate/src/CDD_I2c_PBcfg.c
- _MCAL_generate/src/CDD_I2c_Cfg.c
- _MCAL_generate/include/CDD_I2c_Cfg.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h

- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h

Det module files:

- Det.h

Rte module files:**Dem module files:**

- Dem.h

2.3 Add Plug-ins

I2C module plug-ins are developed for EB tresos Studio, so, to use I2C plug-ins on the EB tresos Studio, the user needs to add the I2C module to the EB plug-ins folder first.

- 1) Copy the I2C module(_MCAL/EB_Plugins/eclipse/plugins/I2c) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
- 2) Set the I2C module output location folder for the generated source file and header files.
- 3) Use EB tresos to configure the I2C module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
I2C_START_SEC_CONFIG_DATA_8 I2C_STOP_SEC_CONFIG_DATA_8 I2C_START_SEC_CONFIG_DATA_16 I2C_STOP_SEC_CONFIG_DATA_16 I2C_START_SEC_CONFIG_DATA_32 I2C_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).
I2C_START_SEC_CONFIG_DATA_UNSPECIFIED I2C_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
I2C_START_SEC_CONST_BOOLEAN I2C_STOP_SEC_CONST_BOOLEAN I2C_START_SEC_CONST_8 I2C_STOP_SEC_CONST_8 I2C_START_SEC_CONST_16 I2C_STOP_SEC_CONST_16 I2C_START_SEC_CONST_32 I2C_STOP_SEC_CONST_32 I2C_START_SEC_CONST_UNSPECIFIED I2C_STOP_SEC_CONST_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
I2C_START_SEC_CODE I2C_STOP_SEC_CODE I2C_START_SEC_RAMCODE I2C_STOP_SEC_RAMCODE I2C_START_SEC_CODE_AC I2C_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
I2C_START_SEC_VAR_NO_INIT_BOOLEAN I2C_STOP_SEC_VAR_NO_INIT_BOOLEAN I2C_START_SEC_VAR_NO_INIT_8 I2C_STOP_SEC_VAR_NO_INIT_8 I2C_START_SEC_VAR_NO_INIT_16 I2C_STOP_SEC_VAR_NO_INIT_16 I2C_START_SEC_VAR_NO_INIT_32 I2C_STOP_SEC_VAR_NO_INIT_32 I2C_START_SEC_VAR_NO_INIT_UNSPECIFIED I2C_STOP_SEC_VAR_NO_INIT_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
I2C_START_SEC_VAR_INIT_BOOLEAN I2C_STOP_SEC_VAR_INIT_BOOLEAN I2C_START_SEC_VAR_INIT_8 I2C_STOP_SEC_VAR_INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by

Section Name	Section Type	Description
I2C_START_SEC_VAR_INIT_16		startup code (data).
I2C_STOP_SEC_VAR_INIT_16		
I2C_START_SEC_VAR_INIT_32		
I2C_STOP_SEC_VAR_INIT_32		
I2C_START_SEC_VAR_INIT_UNSPECIFIED		
I2C_STOP_SEC_VAR_INIT_UNSPECIFIED		

Chapter 4 Exclusive Area

I2C module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the I2c driver:

- CDD_I2c_Hw.c:
 - FCIIIC_Init_Master : exclusive area 0
 - FCIIIC_DeInit_Master : exclusive area 1
 - FCIIIC_Init_Slave : exclusive area 2
 - FCIIIC_DeInit_Slave : exclusive area 3
 - FCIIIC_GetChannelStatus : exclusive area 4
 - FCIIIC_MasterSyncSend : exclusive area 5
 - FCIIIC_MasterSyncReceive : exclusive area 6
 - FCIIIC_MasterAsyncSend : exclusive area 7
 - FCIIIC_MasterAsyncReceive : exclusive area 8
 - FCIIIC_SlaveListening : exclusive area 9
 - FCIIIC_LL_IRQnHandler : exclusive area 10
 - FCIIIC_LL_DMA_IRQnHandler : exclusive area 11
 - FCIIIC_MasterErrorHandler : exclusive area 12
 - FCIIIC_MasterInterruptProcessStop : exclusive area 13
 - FCIIIC_MasterInterruptProcessTransmit : exclusive area 14
 - FCIIIC_MasterInterruptProcessReceive : exclusive area 15
 - FCIIIC_MasterInterruptProcessing : exclusive area 16
 - FCIIIC_SlaveInterruptProcessAddressMatch : exclusive area 17
 - FCIIIC_SlaveInterruptProcessReceive : exclusive area 18
 - FCIIIC_SlaveInterruptProcessTransmit : exclusive area 19
 - FCIIIC_SlaveInterruptProcessStop : exclusive area 20
 - FCIIIC_SlaveInterruptProcessing : exclusive area 21
 - FCIIIC_MasterDmaInit : exclusive area 22
 - FCIIIC_MasterDMAStart : exclusive area 23
 - FCIIIC_MasterDMAStop : exclusive area 24
 - FCIIIC_MasterDMATransmit : exclusive area 25
 - FCIIIC_MasterDMAReceive : exclusive area 28

- FCIIIC_MasterDMAStageData : exclusive area 29
- FCIIIC_MasterDMAStageStop : exclusive area 30
- FCIIIC_MasterDMAStageIdle : exclusive area 26

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
FCI2C0	FCIIC0_IRQHandler	92
FCI2C1	FCIIC1_IRQHandler	93

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
I2c_Init	I2C_E_ALREADY_INITIALIZED; I2C_E_PARAM_POINTER
I2c_DeInit	I2C_E_UNINIT
I2c_SyncTransmit	I2C_E_UNINIT; I2C_E_PARAM_POINTER; I2C_E_PARAM_UNIT; I2C_E_INV_CTRL_IDX
I2c_AsyncTransmit	I2C_E_UNINIT; I2C_E_PARAM_POINTER; I2C_E_PARAM_UNIT; I2C_E_INV_CTRL_IDX
I2c_StartListening	I2C_E_UNINIT; I2C_E_PARAM_POINTER; I2C_E_PARAM_UNIT; I2C_E_INV_CTRL_IDX
I2c_GetStatus	I2C_E_UNINIT; I2C_E_PARAM_UNIT; I2C_E_INV_CTRL_IDX
I2c_GetVersionInfo	I2C_E_PARAM_POINTER

6.2 Dem

None

Chapter 7 Function Calls

7.1 Function Calls during Startup

The API needs be called is `I2c_Init(const I2c_ConfigType *ConfigPtr);`

7.2 Function Calls during Shutdown

None

7.3 Function Calls during Wake-up

None

7.4 Function Calls during Runtime

None

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

- Notification

If the user configures notifications and the value is not "NULL_PTR" or "NULL", the user shall implement the notification function in the user code.

- Callback

None

- Callout

None

8.2 Macros

Please check various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:

AUTOSAR_OS_NOT_USED need defined.

In this case, If USE_SW_VECTOR_MODE is not defined, the user needs to map the interrupt vector table function to ISR function, for example:

```
extern ISR(FCI2C_0_ISR);

void FCIIC0_IRQHandler(void)
{
    FCI2C_0_ISR();
}
```

- If OS is used:

Do not define AUTOSAR_OS_NOT_USED.

Chapter 9 Integration Steps

- 1) Configure the I2C module and generate configuration files (please refer to Building chapter for details).
- 2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
- 4) Build the I2C module with dependent modules.